



EXPERIMENT 6

Student Name: Ridhima Gulati

Branch: AIT-CSE (Cyber Security)

Semester: 5

Subject Name: ADBMS

UID: 24BCY70132

Section/Group: 24AIT_KRG-G2

Date: 10-08-2026

Subject Code: 24CSP-333

EXPERIMENT:

Advanced SQL Queries

AIM: To study and implement advanced SQL queries using conditional logic, aggregate functions, joins, self-joins, and date operations to solve real-world database problems.

Question 1: A fruit distribution company records the number of apples and oranges sold each day. For every date, exactly one record exists for apples and one record exists for oranges.

Generate a daily sales comparison report showing the difference between the number of apples sold and the number of oranges sold on the same day. The difference should be calculated as:

Difference = Apples Sold - Oranges Sold

Display the sale date and the calculated difference. The result should be sorted by sale date in ascending order.

Table:

```
CREATE TABLE Sales (  
    sale_date DATE, fruit  
    VARCHAR(20),  
    sold_num INT,  
  
    PRIMARY KEY (sale_date, fruit)  
);
```

```
INSERT INTO Sales  
VALUES
```

EXPERIMENT 6

```
('2020-05-01','apples',10),
('2020-05-01','oranges',8),
```

```
('2020-05-02','apples',15),
('2020-05-02','oranges',15),
```

```
('2020-05-03','apples',20),
('2020-05-03','oranges',0),
```

```
('2020-05-04','apples',15),
('2020-05-04','oranges',16);
```

```
select * from Sales;
```

Data Output Messages Notifications			
	sale_date [PK] date	fruit [PK] character varying (20)	sold_num integer
1	2020-05-01	apples	10
2	2020-05-01	oranges	8
3	2020-05-02	apples	15
4	2020-05-02	oranges	15
5	2020-05-03	apples	20
6	2020-05-03	oranges	0
7	2020-05-04	apples	15
8	2020-05-04	oranges	16

Answer: SELECT
 SALE_DATE,
 SUM(CASE WHEN
 fruit='apples' then
 sold_num else
 -sold_num end)AS
 DIFFERENCE
 FROM sales
 Group by sale_date
 order by sale_date;

EXPERIMENT 6

Output:

Data Output			Messages	Notifications
			SQL	
	sale_date date	diff integer		
1	2020-05-01	2		
2	2020-05-02	0		
3	2020-05-03	20		
4	2020-05-04	-1		

Question 2: A company stores hierarchical employee reporting information in a table. Each record represents a node in a hierarchy where:

- id represents the current node.
- p_id represents the parent node.

Classify every node into one of the following categories:

- Root: A node that does not have any parent.
- Leaf: A node that has a parent but does not have any children.
- Inner: A node that has both a parent and at least one child.

Display the node ID along with its category. Return the result sorted by node ID.

Table: CREATE TABLE Tree (

id INT PRIMARY KEY,

p_id INT

);

EXPERIMENT 6

INSERT INTO Tree

VALUES (1,NULL),

(2,1),

(3,1),

(4,2),

(5,2);

select * from Tree;

Data Output Messages Notifications			
≡+ 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈 SQL			
	id [PK] integer	p_id integer	
1	1	[null]	
2	2	1	
3	3	1	
4	4	2	
5	5	2	

Answer:SELECT

t.id,

CASE

WHEN t.p_id IS NULL THEN

'Root'

WHEN NOT EXISTS (

SELECT 1

FROM tree c

WHERE c.p_id = t.id

) THEN 'Leaf'

ELSE 'Inner'

END AS Type

FROM tree t

ORDER BY t.id;

EXPERIMENT 6

Output:

Data Output		Messages	No
≡+	📄	▼	🗑️
	id [PK] integer	case text	
1	1	Root	
2	2	Inner	
3	3	Leaf	
4	4	Leaf	
5	5	Leaf	

LEARNING OUTCOMES:

1. I learned to use self-joins to compare records from the same table and calculate the difference between apples and oranges sold on the same date.
2. Gained practical understanding of using JOIN conditions and WHERE clauses to match related records based on common attributes such as sale dates and fruit types.
3. Learned to use the CASE statement to classify hierarchical data into Root, Inner, and Leaf nodes based on parent-child relationships.
4. Developed an understanding of subqueries to identify nodes that act as parents and determine whether a node has child nodes.
5. Enhanced the ability to combine JOINS, CASE expressions, subqueries, conditional logic, and ORDER BY to solve real-world database problems involving sales analysis and hierarchical data.